

Prompt:

8/17/2026, 8:00:47 PM

aku ingin bertanya lebih lanjut mengenai quantitative trading yang semestinya dilakukan oleh hedge fund dan prop firm tapi dengan matematika yang lebih lanjut. maksudku adalah optimasi yang kasusnya TSP, maxcut, portofolio optimization, dst. tapi digunakan untuk trading jangka pendek? mungkin bayanganku seperti ini

1. menggunakan data training harga histori aset yang ingin ditradingkan dan beberapa harga aset yang berhubungan seperti jika ingin membeli altcoin dengan marketcap menengah, maka harus melihat harga btc dan eth (bertujuan untuk menentukan money flow dari aset-aset yang berkaitan)
2. hasil data training digunakan untuk meletakkan order book (tidak langsung eksekusi) berikut TP dan SL sesuai dengan algoritma yang sudah ada agar tetap mengikuti resiko \$1
3. setiap jam atau periode tertentu, posisi dievaluasi apakah masih bisa dipertahankan jika order book belum terpanggil? atau jika order book sudah terpanggil, ada 2 kondisi : jika floating loss, biarkan saja tapi jika floating profit, apakah setup nya layak dipertahankan atau harus setting ulang stop loss nya
4. jika stop loss sudah diletakkan di BEP atau titik tertentu sesuai dengan algoritma dan ternyata masih floating profit tinggi, mungkin bisa dipertimbangan TP nya dinaikkan lagi

Response:

8/17/2026, 8:00:48 PM

Personalizing

Personalized quantitative trading plans around Binance automation and risk controls

Worked for 5s

Ya. Dan menurutku bayanganmu **sudah mengarah ke sesuatu yang jauh lebih menarik daripada sekadar "bot trading dengan indikator"**.

Yang kamu gambarkan sebenarnya bisa diformulasikan sebagai **short-horizon quantitative trading system dengan beberapa layer optimasi**:

Market State → Forecast → Combinatorial Optimization → Order Placement → Risk Control → Dynamic Re-op

Namun ada satu koreksi penting: **TSP, MaxCut, dan QUBO tidak harus dipaksakan langsung menjadi "algoritma untuk menentukan buy/sell"**. Justru penggunaan yang lebih natural adalah mengubah bagian-bagian tertentu dari masalah trading menjadi masalah optimasi kombinatorial.

Dan menurutku, kalau kita ingin membangun ini di atas `TradeProject`, arsitektur yang bagus adalah seperti berikut.

1. Gambaran sistem yang kamu bayangkan

Misalnya kita ingin trading:

$$X = \text{SOL}$$

tetapi tidak hanya menggunakan data SOL.

Kita masukkan:

$$\mathbf{x}_t = [P_{\text{SOL}}, P_{\text{BTC}}, P_{\text{ETH}}, V_{\text{SOL}}, V_{\text{BTC}}, V_{\text{ETH}}, \text{OB}_{\text{SOL}}, \text{OB}_{\text{BTC}}, \text{OB}_{\text{ETH}}, \dots]$$

Jadi keadaan pasar bukan:

"SOL naik atau turun?"

melainkan:

"Bagaimana keadaan SOL relatif terhadap ekosistem aset yang memengaruhinya?"

Kemudian sistem melakukan:

Historical Market Data

|



Feature / State Construction

|

|— SOL

|— BTC

|— ETH

|— volume

|— volatility

|— order book

|— correlations

|



Short-Term Forecast

|



Optimization Problem

|

|— Entry price

|— Order type

|— Position size

|— SL

|— TP

|— Exit policy

|



Pending Order

|



Market Monitoring

|



Re-optimization

|

|— Cancel

|— Keep

|— Move SL

|— Move TP

|— Close

Ini sudah jauh lebih dekat dengan cara berpikir systematic trading daripada sekadar strategi teknikal.

2. Bagian pertama yang sangat penting: jangan hanya menggunakan harga aset target

Contohmu tentang altcoin + BTC + ETH itu tepat.

Misalnya kita ingin memprediksi return SOL pada horizon 1 jam:

$$r_{\text{SOL}, t \rightarrow t+1h}$$

Daripada hanya:

$$r_{\text{SOL}, t}$$

kita bisa menggunakan:

$$\mathbf{r}_t = \begin{bmatrix} r_{\text{SOL}, t} \\ r_{\text{BTC}, t} \\ r_{\text{ETH}, t} \\ r_{\text{BNB}, t} \\ \vdots \end{bmatrix}$$

Kemudian kita dapat membangun model:

$$\hat{r}_{\text{SOL}, t+1} = f(\mathbf{x}_t)$$

Tetapi yang menarik adalah kita tidak harus berhenti pada korelasi Pearson.

Kita dapat membangun **conditional dependence**.

Misalnya:

$$\text{Corr}(r_{\text{SOL}}, r_{\text{BTC}})$$

tetapi juga:

$$\text{Corr}(r_{\text{SOL}}, r_{\text{BTC}} \mid r_{\text{ETH}})$$

atau bahkan mutual information:

$$I(X; Y) = \sum_{x, y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

Ini mulai masuk ke wilayah **information-theoretic quantitative finance**.

3. Bahkan "money flow" yang kamu maksud dapat dibuat matematis

Misalnya BTC bergerak:

+2%

ETH:

+3%

sementara SOL:

+0.2%

Sistem dapat melihat:

BTC dan ETH sudah bergerak, tetapi SOL belum mengikuti.

Namun jangan langsung menganggap:

"SOL pasti naik."

Lebih tepat:

$$E[r_{\text{SOL}} \mid r_{\text{BTC}}, r_{\text{ETH}}, V, \sigma, \text{order book}]$$

Kemudian kita mencari:

$$\mu_t = E[r_{t+\Delta t} \mid F_t]$$

dan:

$$\sigma_t^2 = \text{Var}(r_{t+\Delta t} \mid F_t)$$

Jadi setiap keputusan memiliki dua komponen:

$$\text{Expected Return} + \text{Expected Risk}$$

4. Lalu masuk ke optimasi

Nah, di sinilah ide TSP/MaxCut/QUBO yang kamu sebut menjadi relevan.

Misalkan kita memiliki 10 kandidat aset:

$$A = \{A_1, \dots, A_{10}\}$$

dan hanya boleh memilih 3.

Kita dapat mendefinisikan binary variable:

$$x_i \in \{0, 1\}$$

dengan:

$$x_i = \begin{cases} 1 & \text{aset dipilih} \\ 0 & \text{tidak} \end{cases}$$

Constraint:

$$\sum_i x_i = 3$$

Kemudian objective:

$$\max_x \left[\sum_i \mu_i x_i - \lambda \sum_{i < j} \Sigma_{ij} x_i x_j \right]$$

Ini sudah menjadi **binary portfolio optimization**.

Dan dapat ditulis sebagai QUBO:

$$\boxed{\min_x x^T Q x}$$

dengan penalty:

$$P \left(\sum_i x_i - K \right)^2$$

untuk memastikan jumlah aset = K .

Ini sangat natural untuk:

- QAOA
- simulated annealing
- classical branch-and-bound
- tabu search
- quantum annealing

dan sebagainya.

5. Tetapi untuk short-term trading, optimasinya bisa lebih menarik

Daripada hanya:

"aset mana yang dibeli?"

kita bisa membuat decision variable:

$$X_{i,p,s}$$

dengan:

- i = aset
- p = price level
- s = side

Contohnya:

$$X_{\text{SOL},p,\text{long}}$$

berarti:

pasang limit buy SOL pada harga p .

Kemudian:

$$X_{\text{SOL},p,\text{short}}$$

berarti:

pasang short pada harga p .

Sekarang masalahnya berubah menjadi:

Di mana saya harus menaruh order?

Ini jauh lebih dekat dengan yang kamu bayangkan.

6. Order book sebenarnya dapat dijadikan optimization landscape

Misalnya order book:

```
ASK
101.0    20 SOL
100.9    15 SOL
100.8    50 SOL
-----
100.7    ← current
-----
100.6    10 SOL
100.5    35 SOL
100.4    80 SOL
BID
```

Kita dapat membuat candidate entry:

$$p \in \{100.4, 100.5, 100.6\}$$

Untuk setiap p , hitung:

$$P(\text{fill} \mid p)$$

dan:

$$E[R \mid p]$$

serta:

$$P(\text{adverse move} \mid p)$$

Kemudian:

$$p^* = \arg \max_p E[\text{utility} \mid p]$$

Ini sudah menjadi **optimal execution / order placement problem**.

Dan menurutku ini jauh lebih cocok untuk matematika lanjutan daripada sekadar membuat RSI yang lebih rumit.

7. Constraint \$1 yang kamu sebut sangat bagus

Kamu mengatakan:

tetap mengikuti risiko \$1.

Kita bisa menjadikannya hard constraint.

Misalnya:

$$R_{\max} = \$1$$

Jika entry:

$$P_e$$

dan stop:

$$P_{SL}$$

maka:

$$N = \frac{1}{|P_e - P_{SL}|}$$

sehingga:

$$N|P_e - P_{SL}| \leq 1$$

Ini sangat penting.

Karena sekarang optimizer **tidak boleh mencari solusi yang bagus tetapi mempertaruhkan \$10 atau \$100**.

Risk constraint menjadi bagian dari mathematical optimization problem.

8. Bahkan TP dan SL dapat menjadi variable optimasi

Ini menurutku salah satu bagian terbaik dari ide kamu.

Jangan menetapkan:

$$SL = 2\%$$

dan:

$$TP = 3\%$$

secara statis.

Anggap:

$$s = \text{stop distance}$$

$$t = \text{take-profit distance}$$

Kita ingin mencari:

$$(s^*, t^*) = \arg \max_{s, t} E[U(s, t)]$$

dengan constraint:

$$\text{Expected Loss} \leq \$1$$

dan mungkin:

$$P(\text{TP hit})$$

diperoleh dari historical distribution.

Misalnya:

SL	TP	P(TP)	P(SL)	Expected PnL
0.5%	0.8%	0.70	0.30	+...
0.5%	1.2%	0.52	0.48	+...
1.0%	1.5%	0.48	0.52	+...
1.5%	3.0%	0.32	0.68	+...

Optimizer memilih kombinasi terbaik **berdasarkan kondisi pasar saat itu.**

9. Kemudian masuk ke ide kamu nomor 3

Ini menurutku sangat bagus.

Misalnya order:

$$O_0 = (P_e, P_{SL}, P_{TP}, N)$$

dipasang pada:

$$t_0$$

Tetapi pada:

$$t_1 = t_0 + 1h$$

order belum filled.

Kita jangan bertanya:

"Apakah waktunya sudah 1 jam?"

Tetapi:

$$P(\text{fill} \mid F_{t_1})$$

dan:

$$E[U(\text{keep})]$$

dibandingkan dengan:

$$E[U(\text{cancel and re-optimize})]$$

Jadi keputusan:

Keep	vs	Cancel + Re-optimize
------	----	----------------------

sendiri merupakan optimization problem.

10. Kalau sudah filled, problem-nya berubah

Ini penting.

Sebelum filled:

Optimal Entry Problem

Setelah filled:

Optimal Position Management Problem

Misalnya posisi long.

Kondisi:

$$P_t < P_e$$

→ floating loss.

Sedangkan:

$$P_t > P_e$$

→ floating profit.

Tetapi **jangan otomatis mengatakan floating loss harus dibiarkan dan floating profit harus dikelola.**

Yang lebih matematis adalah:

$$E[V_{\text{hold}} | F_t]$$

dibandingkan:

$$V_{\text{close}}$$



Jadi:

$$\text{Hold if } E[V_{\text{future}} | F_t] > V_{\text{close}}$$

11. Ini sebenarnya merupakan sequential decision problem

Dan ini menurutku adalah bagian paling penting dari seluruh ide kamu.

Trading bukan:

Buy → Wait → Sell

tetapi:

$$a_t \rightarrow s_{t+1} \rightarrow a_{t+1} \rightarrow s_{t+2} \rightarrow \dots$$

dengan:

s_t = market state

dan:

a_t = trading action

Misalnya:

$$a_t \in \{\text{cancel, keep, enter, move SL, move TP, close}\}$$

Ini sudah mulai menyerupai:

Markov Decision Process

atau bahkan:

Model Predictive Control (MPC)

12. Dan MPC menurutku sangat cocok dengan ide kamu

Bayangkan setiap jam sistem melakukan:

State estimation

$$s_t = [P_t, V_t, \sigma_t, r_{\text{BTC}}, r_{\text{ETH}}, OB_t, \mu_t, \Sigma_t, \text{position}]$$

Kemudian optimizer menghitung:

$$a_t^* = \arg \max_a E[U(s_{t+1}) \mid s_t, a]$$

Lalu hanya menjalankan keputusan pertama.

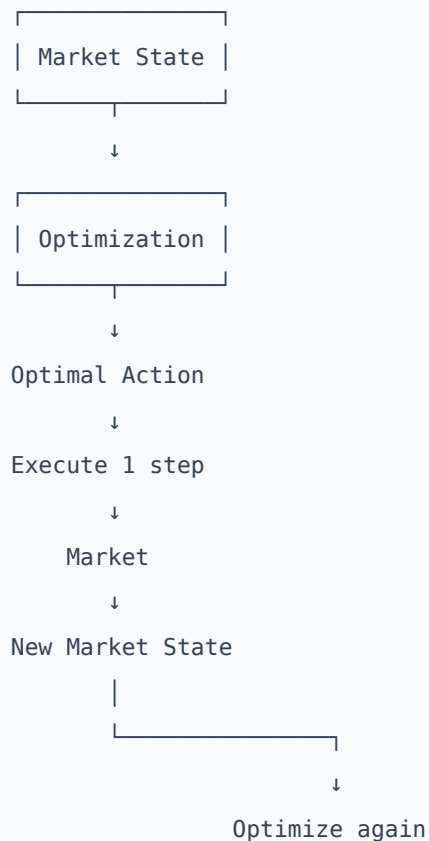
Satu jam kemudian:

$$s_{t+1}$$

berubah.

Optimizer dijalankan lagi.

Ini adalah prinsip **receding horizon control**.



Ini sangat dekat dengan apa yang kamu gambarkan sebagai **evaluasi setiap periode**.

13. Sekarang kasus floating profit yang kamu sebut

Misalnya:

$$P_e = 100$$

kemudian:

$$P_t = 105$$

dan SL awal:

$$SL = 98$$

Kita tidak sekadar berkata:

"Profit 5%, geser SL ke 100."

Kita dapat menghitung ulang distribusi return:

$$p(r_{t+\Delta t} | F_t)$$

Misalnya ternyata:

$$E[r_{t+\Delta t}] = +1.8\%$$

dengan volatilitas:

$$\sigma = 0.7\%$$

maka probabilitas continuation mungkin masih cukup tinggi.

Tetapi jika:

$$E[r_{t+\Delta t}] = -0.2\%$$

maka momentum sudah melemah.

Jadi keputusan trailing stop dapat menjadi:

$$SL_t^* = \arg \max_{SL} E[U(SL) | F_t]$$

14. Bahkan TP dapat dinaikkan secara optimal

Ini persis ide nomor 4 milikmu.

Misalnya:

$$P_e = 100$$

$$TP = 105$$

Harga sekarang:

$$P_t = 104$$

Kemudian model memperkirakan:

$$P(P_{t+1} > 105) = 0.72$$

dan:

$$P(P_{t+1} > 108) = 0.51$$

Maka kita dapat membandingkan:

$$TP = 105$$

versus:

$$TP = 108$$

dan menghitung:

$$E[U(TP = 105)]$$

versus:

$$E[U(TP = 108)]$$

Kalau:

$$E[U(108)] > E[U(105)]$$

maka TP dinaikkan.

Jadi:

$$TP_t \rightarrow TP_{t+1} \rightarrow TP_{t+2}$$

secara dinamis.

15. Nah, di sinilah TSP/MaxCut/QAOA mulai bisa masuk

Kita sekarang mempunyai banyak kandidat keputusan.

Misalnya:

Candidate actions

$$A = \left\{ \begin{array}{l} \text{Keep TP 105} \\ \text{Raise TP 106} \\ \text{Raise TP 108} \\ \text{Move SL to 101} \\ \text{Move SL to 102} \\ \text{Close position} \end{array} \right\}$$

Kita dapat mendefinisikan binary variables:

$$x_i \in \{0, 1\}$$

dengan constraint:

$$\sum_i x_i = 1$$

dan objective:

$$\max_x \sum_i U_i x_i - \lambda \sum_{i < j} C_{ij} x_i x_j$$

Ini menjadi QUBO.

Kemudian:

bisa menjadi salah satu solver.

16. Tapi jangan gunakan QAOA untuk semuanya

Ini sangat penting kalau kamu ingin membuat proyek yang benar-benar serius.

Misalnya:

"Saya menggunakan QAOA untuk menentukan apakah BTC akan naik."

Menurutku itu **bukan penggunaan terbaik**.

Lebih masuk akal:

Statistical/ML Model → Forecast

kemudian:

QUBO/QAOA → Decision Optimization

Jadi:

Classical model

menjawab:

Apa yang kemungkinan terjadi?

Sedangkan:

Optimization model

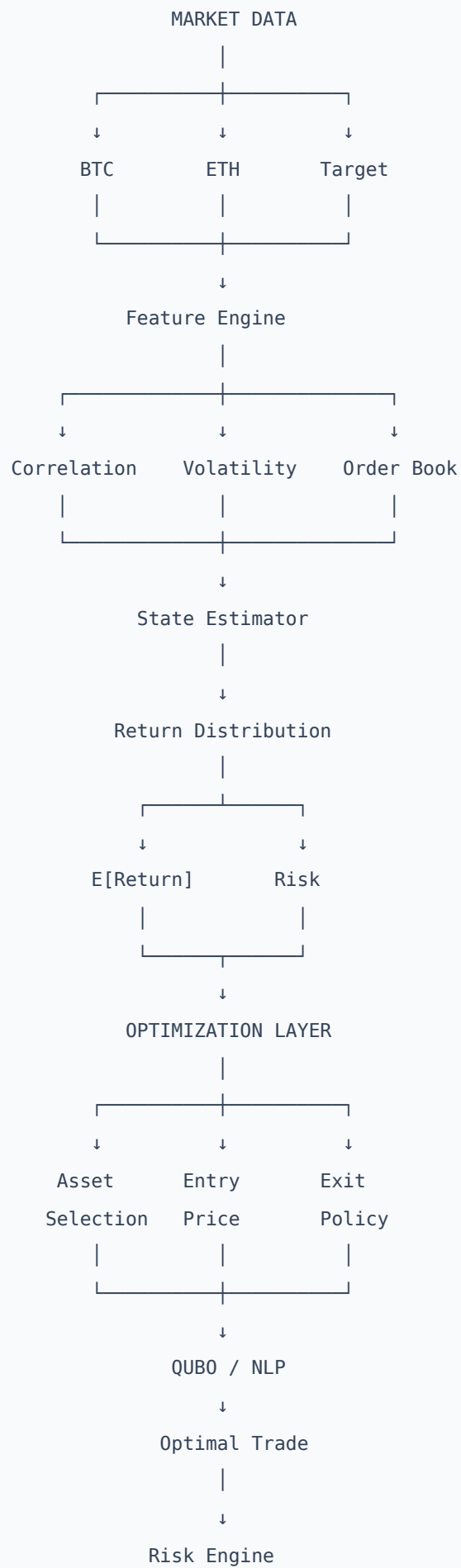
menjawab:

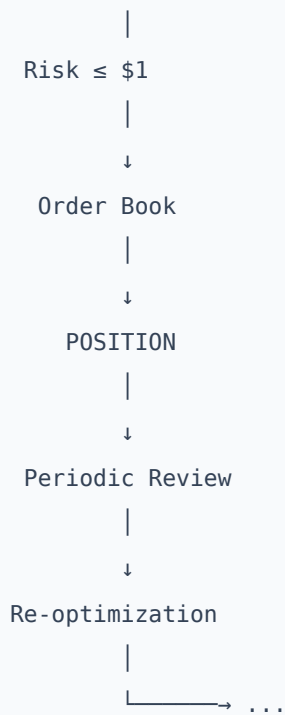
Dengan kemungkinan tersebut, keputusan terbaik apa yang dapat saya ambil?

Ini pemisahan yang sangat penting.

17. Bahkan arsitektur akhirnya bisa seperti ini

Menurutku ini jauh lebih kuat:





18. Kalau dikaitkan langsung dengan TradeProject

Repository yang sekarang sebenarnya sudah memiliki fondasi:

Market Data → Strategy → Position Sizer → Execution

Yang kita perlukan adalah menambahkan satu layer besar:

Optimization Engine

sehingga:

```
TradeProject
|
├─ Data
|
├─ Forecasting
|
├─ Strategies
|
├─ Optimization      ← layer baru
|   └─ Portfolio
|   └─ Entry
|   └─ Order placement
|   └─ SL/TP
|   └─ Exit
|
├─ Risk
|
├─ Execution
|
└─ Backtesting
```

Dan yang paling penting: **backtester harus bisa mensimulasikan seluruh siklus ini**, bukan hanya menghitung apakah signal menghasilkan profit.

19. Ada satu masalah besar yang harus kita selesaikan: look-ahead bias

Kalau kita melakukan ini dengan historical data, bahaya terbesar bukan QAOA, bukan VQE, dan bukan optimizer.

Tetapi:

look-ahead bias

Misalnya pada:

$t = 10 : 00$

sistem memutuskan order.

Ia hanya boleh mengetahui:

$$F_{10:00}$$

Tidak boleh mengetahui:

$$P_{10:30}$$

walaupun data tersebut tersedia dalam dataset historis.

Jadi simulasi harus benar-benar:

$$\boxed{\text{Past} \rightarrow \text{Decision} \rightarrow \text{Future}}$$

bukan:

$$\text{Past} + \text{Future} \rightarrow \text{Decision}$$

20. Masalah kedua: order book jauh lebih sulit daripada OHLCV

Kalau kita benar-benar ingin melakukan pendekatan yang kamu bayangkan, aku justru menyarankan **jangan langsung mulai dari OHLCV saja**.

Karena untuk short-term trading:

OHLCV

tidak cukup untuk mengetahui:

- bid/ask imbalance,
- queue position,
- spread,
- depth,
- liquidity,
- slippage,
- fill probability,
- market impact.

Kita perlu mempertimbangkan:

$$OB_t = \{P_i^{bid}, Q_i^{bid}, P_i^{ask}, Q_i^{ask}\}$$

Misalnya order-book imbalance:

$$I_t = \frac{V_{\text{bid}} - V_{\text{ask}}}{V_{\text{bid}} + V_{\text{ask}}}$$

Kemudian:

$$I_t > 0$$

memberikan indikasi tekanan beli.

Tetapi lagi-lagi, **indikasi ≠ prediksi pasti**.

21. Jadi kalau aku harus mendefinisikan proyek ini secara akademis

Aku tidak akan menyebutnya:

"AI trading bot."

Aku akan mendefinisikannya sebagai:

Short-Horizon Systematic Trading via Dynamic Mathematical Optimization

dengan komponen:

Layer 1 — Market State

$$F_t$$

Layer 2 — Statistical Forecasting

$$p(r_{t+\Delta t} \mid F_t)$$

Layer 3 — Combinatorial Optimization

$$x^* = \arg \min_x x^T Q x$$

Layer 4 — Risk-Constrained Execution

$$\text{Risk}(x) \leq \$1$$

Layer 5 — Sequential Re-optimization

$$x_{t+1}^* = \arg \max_x E[U(x) | F_{t+1}]$$

Dan ini menghasilkan sesuatu yang menurutku sangat menarik

Kamu sebenarnya sedang menggabungkan **tiga bidang**:

Quantitative Finance + Operations Research + Quantum Optimization

Bukan:

Quantum Computing → prediksi harga

tetapi:

Market Modeling → Optimization → Execution
--

Dan ini juga membuat pengalamanmu dengan **Markowitz → QUBO → Ising → VQE/QAOA** menjadi sangat relevan. Portfolio optimization yang kamu kerjakan untuk skripsi sebenarnya bisa menjadi **versi long-horizon/static** dari problem yang sekarang ingin kamu bawa menjadi **short-horizon/dynamic optimization**.

Perbedaannya kira-kira:

Skripsi	→	Trading System
Asset allocation		Order/action allocation
Expected return		Short-term forecast
Covariance		Dynamic risk
QUBO		Dynamic QUBO
VQE		QAOA / hybrid solver
One optimization		Repeated optimization

Menurutku, justru bagian "repeated optimization" inilah yang membuat proyek ini jauh lebih menarik.

Langkah berikut yang paling masuk akal adalah kita **jangan langsung coding**. Kita desain dulu *mathematical formulation* untuk satu kasus konkret, misalnya **BTC sebagai market factor + ETH sebagai related asset + satu altcoin target**, lalu kita turunkan dari data historis → probabilitas return → optimal limit-entry → SL/TP → \$1 risk constraint → evaluasi 1 jam → re-optimization. Setelah formulasi klasiknya benar, baru kita tentukan **bagian mana yang benar-benar cocok dijadikan QUBO/Ising/QAOA dan bagian mana yang lebih baik diselesaikan secara classical**.